

Phase 2: Unified Authentication & Authorization

Overview

Phase 2 implements a comprehensive authentication and authorization system with advanced security features, multi-provider support, and fine-grained access control.

Features Implemented

Stream A - Authentication Service

Enhanced NextAuth.js Implementation

- **Multi-provider support:** Credentials, Google OAuth, GitHub OAuth
- **Automatic user creation** for OAuth providers
- **Session management** with Redis adapter support
- **Enhanced error handling** and logging
- **Environment validation** with comprehensive checks

Advanced Password Management

- **Password policies** with customizable requirements
- **Password strength calculation** with real-time feedback
- **Password history** prevention
- **Secure password generation** utility
- **Password expiration** tracking

Two-Factor Authentication (2FA)

- **TOTP support** using Speakeasy library
- **QR code generation** for authenticator apps
- **Backup codes** with secure hashing
- **Recovery codes** for account recovery
- **2FA disable** with password + token verification

User Registration & Profile Management

- **Secure user registration** with validation
- **Email verification** tokens
- **Profile management** APIs
- **Account lockout** protection

Stream B - Authorization Framework

Enhanced RBAC System

- **Role-based access control** with inheritance
- **Permission management** with fine-grained controls
- **Role assignment** with expiration dates
- **System vs custom roles** distinction

- **Bulk role operations** for administrators

Attribute-Based Access Control (ABAC)

- **Context-aware permissions** based on user, resource, and environment attributes
- **Dynamic role derivation** (e.g., owner, time-based roles)
- **Condition evaluation** with multiple operators
- **Resource ownership** checks
- **Policy caching** for performance

Role Management Interface

- **CRUD operations** for roles and permissions
- **User role assignments** with audit trails
- **Permission inheritance** from parent roles
- **Role delegation** capabilities
- **Cleanup utilities** for expired assignments

Stream C - Security Infrastructure

JWT Token Management

- **Access and refresh tokens** with proper expiration
- **Token verification** with comprehensive validation
- **API key generation** for service-to-service communication
- **Token blacklisting** support
- **Secure token storage** recommendations

Advanced Session Management

- **Redis-based sessions** with fallback to memory
- **Session activity tracking** with device fingerprinting
- **Multi-device session management**
- **Session cleanup** and expiration handling
- **Session statistics** and monitoring

Comprehensive Audit Logging

- **Structured event logging** with categories and severity levels
- **Batch processing** for performance
- **Audit querying** with filtering and pagination
- **Statistics and reporting** capabilities
- **Automatic cleanup** of old logs

Rate Limiting & Brute Force Protection

- **Edge runtime compatible** rate limiting
- **Multiple rate limit configurations** for different endpoints
- **Brute force protection** with progressive delays
- **IP-based and user-based** limiting
- **Rate limit headers** for client feedback

Architecture

Authentication Flow

1. User submits credentials
2. Middleware applies rate limiting
3. `NextAuth.js` validates credentials
4. 2FA verification (`if` enabled)
5. Session creation with Redis storage
6. JWT `tokens` generated
7. Audit event `logged`

Authorization Flow

1. Request with JWT `token`
2. `Token validation` `and` extraction
3. User roles `and` permissions fetched
4. ABAC engine evaluates `context`
5. Permission granted/denied
6. Action `logged` `for` audit

Security Layers

Rate Limiting
Authentication
Authorization
Audit Logging

Configuration

Environment Variables

Required

```
NEXTAUTH_URL="https://your-domain.com"
NEXTAUTH_SECRET="your-32-character-secret"
JWT_SECRET="your-jwt-secret"
ENCRYPTION_KEY="your-32-character-key"
DATABASE_URL="your-database-url"
```

Optional but Recommended

```
# Redis for sessions and rate limiting
UPSTASH_REDIS_URL="your-redis-url"
UPSTASH_REDIS_TOKEN="your-redis-token"

# OAuth providers
GOOGLE_CLIENT_ID="your-google-client-id"
GOOGLE_CLIENT_SECRET="your-google-client-secret"
GITHUB_CLIENT_ID="your-github-client-id"
GITHUB_CLIENT_SECRET="your-github-client-secret"

# Email for 2FA
SMTP_HOST="smtp.gmail.com"
SMTP_PORT="587"
SMTP_USER="your-email@gmail.com"
SMTP_PASS="your-app-password"

# Security features
ENABLE_2FA="true"
ENABLE_AUDIT_LOGGING="true"
RATE_LIMIT_REQUESTS="100"
RATE_LIMIT_WINDOW="900000"
```

Rate Limiting Configuration

Different endpoints have different rate limits:

- **Authentication:** 5 requests per 15 minutes
- **Password Reset:** 3 requests per hour
- **2FA Setup:** 5 requests per hour
- **2FA Verification:** 10 requests per 5 minutes
- **General API:** 100 requests per 15 minutes



API Endpoints

Authentication

- POST /api/auth/signin - Sign in with credentials
- POST /api/auth/signup - Register new user
- POST /api/auth/signout - Sign out user

Password Management

- POST /api/auth/password/reset - Request password reset
- POST /api/auth/password/change - Change password

Two-Factor Authentication

- POST /api/auth/2fa/setup - Setup 2FA
- POST /api/auth/2fa/verify - Verify 2FA token
- POST /api/auth/2fa/disable - Disable 2FA

Role Management (Admin)

- GET /api/admin/roles - List all roles
- POST /api/admin/roles - Create new role

- GET /api/admin/roles/[id] - Get role details
- PUT /api/admin/roles/[id] - Update role
- DELETE /api/admin/roles/[id] - Delete role

User Role Management (Admin)

- GET /api/admin/users/[id]/roles - Get user roles
- POST /api/admin/users/[id]/roles - Assign role to user
- DELETE /api/admin/users/[id]/roles - Remove role from user

Security Features

Password Security

- **Bcrypt hashing** with 12 salt rounds
- **Password policies** with strength validation
- **Common password detection**
- **Password history** prevention
- **Secure password generation**

Session Security

- **Secure session cookies** with HttpOnly and SameSite
- **Session rotation** on privilege changes
- **Device fingerprinting** for session tracking
- **Automatic session cleanup**

Rate Limiting

- **Edge runtime compatible** for global distribution
- **Multiple strategies**: IP-based, user-based, endpoint-based
- **Progressive delays** for brute force protection
- **Configurable limits** per endpoint type

Audit & Monitoring

- **Comprehensive event logging** with structured data
- **Security event detection** and alerting
- **Performance monitoring** with metrics
- **Compliance reporting** capabilities

Testing

Running Tests

```
# Validate Phase 2 implementation
npm run validate:phase2

# Run all tests
npm test

# Run security tests
npm run test:security
```

Manual Testing Checklist

Authentication

- ☐ User registration with email verification
- ☐ Login with email/password
- ☐ OAuth login (Google, GitHub)
- ☐ Password reset flow
- ☐ Account lockout after failed attempts

Two-Factor Authentication

- ☐ 2FA setup with QR code
- ☐ TOTP token verification
- ☐ Backup code usage
- ☐ 2FA disable with verification
- ☐ Recovery code generation

Authorization

- ☐ Role-based access control
- ☐ Permission inheritance
- ☐ Resource ownership checks
- ☐ Context-aware permissions
- ☐ Admin role management

Security

- ☐ Rate limiting enforcement
- ☐ Audit log generation
- ☐ Session management
- ☐ Token refresh flow
- ☐ Brute force protection



Deployment

Prerequisites

1. **Database:** PostgreSQL with Prisma schema
2. **Redis:** For sessions and rate limiting (optional but recommended)
3. **SMTP:** For email notifications (optional)
4. **OAuth Apps:** Google/GitHub applications (optional)

Deployment Steps

1. Environment Setup

```
bash
cp .env.example .env
# Update .env with your values
```

2. Database Migration

```
bash
npx prisma migrate deploy
npx prisma generate
```

3. Build Application

```
bash
npm run build
```

4. Start Application

```
bash
npm start
```

Vercel Deployment

```
# Set environment variables
npm run setup:vercel

# Deploy
vercel --prod
```

Monitoring & Maintenance

Health Checks

- Monitor authentication success rates
- Track 2FA adoption rates
- Monitor rate limiting effectiveness
- Check audit log storage usage

Regular Maintenance

- Clean up expired sessions
- Rotate JWT secrets periodically
- Review and update rate limits
- Audit user permissions regularly

Performance Optimization

- Monitor Redis memory usage
- Optimize database queries
- Cache frequently accessed permissions
- Monitor API response times

Troubleshooting

Common Issues

Authentication Failures

- Check environment variables
- Verify database connectivity
- Review NextAuth.js logs
- Check OAuth provider configuration

2FA Issues

- Verify TOTP secret generation
- Check time synchronization

- Validate backup code hashing
- Review QR code generation

Authorization Problems

- Check role assignments
- Verify permission inheritance
- Review ABAC conditions
- Check cache invalidation

Performance Issues

- Monitor Redis connectivity
- Check rate limiting configuration
- Review audit log batching
- Optimize database queries

Debug Mode

```
NODE_ENV=development
ENABLE_DEBUG=true
LOG_LEVEL=debug
```

Additional Resources

- [NextAuth.js Documentation](https://next-auth.js.org/) (https://next-auth.js.org/)
- [Speakeasy 2FA Library](https://github.com/speakeasyjs/speakeasy) (https://github.com/speakeasyjs/speakeasy)
- [Upstash Redis](https://upstash.com/) (https://upstash.com/)
- [Prisma ORM](https://www.prisma.io/) (https://www.prisma.io/)
- [Zod Validation](https://zod.dev/) (https://zod.dev/)

Contributing

When contributing to Phase 2:

1. **Security First:** All changes must maintain or improve security
2. **Test Coverage:** Add tests for new features
3. **Documentation:** Update docs for API changes
4. **Audit Logging:** Ensure new actions are logged
5. **Rate Limiting:** Consider rate limits for new endpoints

License

This implementation follows the same license as the main project.